



Instituto Infnet

Do outro lado da API

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 11 · Etapa 6 · APIs com FastAPI

Prof. Renan Santos

Onde paramos (Aula 10)

Quatro fontes no painel — a última, uma página dinâmica coletada com Selenium, à parte.

- **O bilhete: 'a fonte do SEU projeto está em qual degrau?'**

Quem ficou no degrau 1 — uma API documentada? Guarde: hoje vocês CONSTROEM um.

- **A escada: API → HTML → JSON por trás → navegador**

Suba um degrau só quando o de baixo não serve — e escreva o motivo.

- **A coleta grava em data/processed/; o app só lê**

É dessa pasta que a API de hoje vai servir os dados.

► Até hoje o painel PEDIU dados. Hoje alguém pede dados A ELE.



Tem como o sistema deles buscar direto?

O pedido da gestora — de uma prefeitura parceira.

- **"A Secretaria de Meio Ambiente quer os números do painel no sistema deles."**

O desmatamento por estado, principalmente — atualizado.

- **"Pediram para não mandarmos planilha por e-mail toda semana."**

Copiar e colar é o que a gestora fazia antes do painel existir.

- **O reflexo da Aula 7: 'eles raspam o nosso painel'**

Antes de concordar, vamos ver o que um programa recebe do painel.

► A pergunta da aula: como deixar OUTRO PROGRAMA usar os nossos dados — sem raspar o painel?

A casca — agora do nosso lado

```
requests.get("http://localhost:8501/desmatamento")
```

```
status 200 · 10.951 bytes
```

```
texto visível: 'Streamlit You need to enable JavaScript  
to run this app.'
```

```
tabelas: 0
```

```
# o corpo do HTML:
```

```
# <noscript>You need to enable JavaScript...</noscript>
```

```
# <div id="root"></div>
```

```
# o NOSSO painel é uma página dinâmica
```

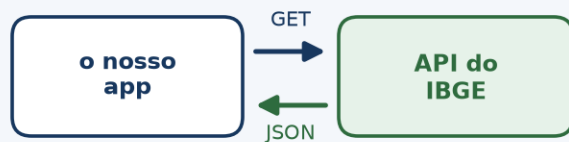
De consumidor a provedor

Subimos a escada para COLETAR. Hoje construímos o degrau 1 para quem quiser os NOSSOS dados.

Do outro lado da API: desde a Aula 2 nós CONSUMIMOS — hoje, OFERECEMOS

Aula 2 · somos CONSUMIDORES

o nosso código pede; a API do IBGE responde



```
requests.get(URL_ESTADOS)
r.json() # lista de UFs
```

degrau 1 · existe contrato escrito

Um programa lendo o NOSSO painel

`requests.get(painel)` → BeautifulSoup

```
status 200 · 10.951 bytes

<noscript>You need to enable
  JavaScript to run this app.
</noscript>
<div id="root"></div>

tabelas: 0
```

para ele, o painel é uma CASCA → degrau 4

Aula 11 · somos PROVEDORES

outro programa pede; a NOSSA API responde



```
GET /desmatamento/PA
{"licenca": "CC BY-SA 4.0",
 "dados": [...]}
```

degrau 1 · documentado em /docs

Subimos a escada para COLETAR. Agora construímos o degrau 1 para quem quiser os NOSSOS dados — sem raspar o painel.

Onde estamos no Projeto de Bloco

Etapa 6, primeira metade — o item 4 do TP3 (a API).



Objetivos da aula

Quatro capacidades — da razão de ter uma API ao primeiro programa que a consome.

- **Explicar o papel da API**

O painel para pessoas; a API para programas — e por que não raspar.

- **Configurar o ambiente (3.3)**

requirements-api.txt, uvicorn, /docs — e os sintomas quando dá errado.

- **Construir rotas GET tipadas (3.4)**

Caminho × consulta; o tipo como guarda; a ordem das rotas.

- **Testar a própria API**

Pelo /docs e por um programa — o código da Aula 2, apontado para nós.

O item 4 do TP3

Leia em voz alta — a frase inteira.

- **"Configure o ambiente de desenvolvimento para a criação de APIs com FastAPI."**
É a subcompetência 3.3 — o primeiro bloco de hoje.
- **"Consulta de dados (GET), envio de novos dados (POST) — ao menos duas rotas."**
Hoje: as consultas. Aula 12: o envio.
- **"... e documente-as adequadamente, explicando sua função e como podem ser usadas."**
O FastAPI gera a documentação do código — mas o que ela diz, quem escreve é você.

► O item avalia rotas que funcionam E estão documentadas. As duas coisas.



1 • Do outro lado

O que vocês já sabem — visto do outro lado

Desde a Aula 2 vocês escrevem o lado do CLIENTE. Cada escolha dele vira uma decisão do SERVIDOR.

- **Cliente escolhe método e caminho → servidor declara as ROTAS**
`requests.get('/localidades/estados') → @app.get('/desmatamento/{uf}')`
- **Cliente passa `params={...}` → servidor declara QUE parâmetros aceita, e de que tipo**
E recusa o que não serve — antes de a função rodar.
- **Cliente lê `.json()` e confere o status → servidor devolve dicionários (viram JSON)**
E decide o que é 200 e o que é recusado.
- **Cliente lê a documentação do IBGE → servidor ESCREVE a sua**
O FastAPI a gera do código: `/docs`.

► **Rota = método + caminho. Handler = a função embaixo do decorador. Endpoint = a rota vista por quem chama.**

Por que FastAPI: os tipos fazem duas coisas

As duas vantagens vêm do mesmo lugar — as anotações de tipo do Python.

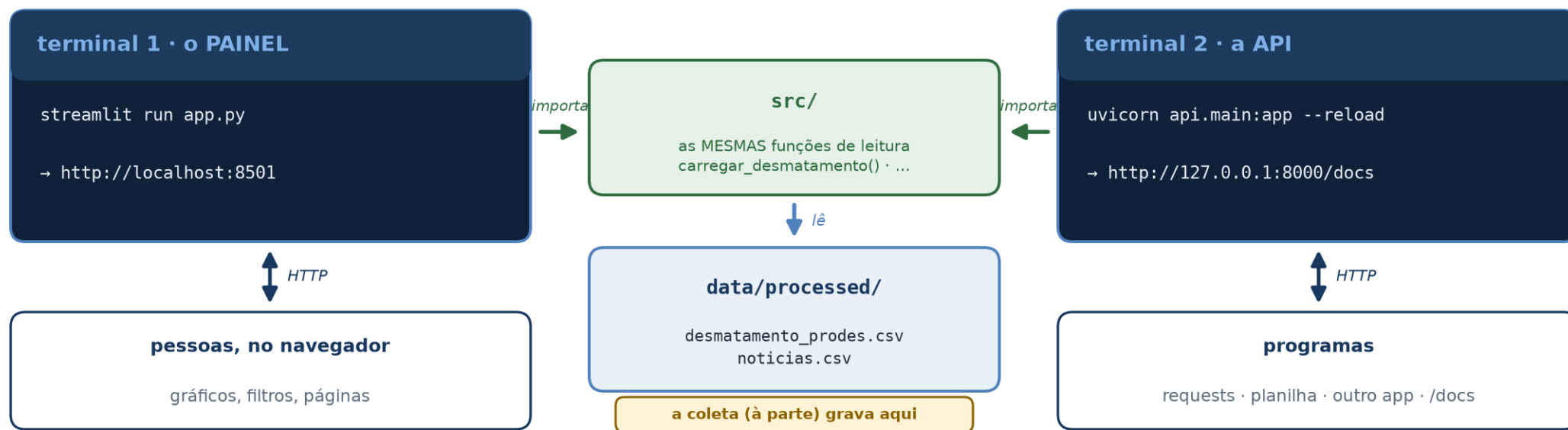
- **Validação: ano: int converte o texto da URL — e recusa o que não converte**
422, com o motivo, ANTES de a sua função rodar. Quem faz é o Pydantic.
- **Documentação: dos mesmos tipos sai o /docs**
'FastAPI... automatically documents our routes... path and query parameters' [Adeshina, p48].
- **Por baixo: Starlette (o servidor web) e uvicorn (quem o roda)**
O Streamlit 1.61 já instala os dois, por dependência própria — já estavam no ambiente.

► **Declarar o tipo não é enfeite: é o contrato — conferido na entrada e publicado no /docs.**

Dois programas, uma pasta

O painel (:8501) e a API (:8000) importam as mesmas funções de `src/` — e leem os mesmos arquivos.

Dois programas, uma pasta: o painel para PESSOAS, a API para PROGRAMAS



Uma leitura, dois clientes. Nenhum dos dois coleta: a coleta roda à parte e grava em `data/processed/`.



2 · O ambiente e a primeira API

Um terceiro arquivo de requisitos

'fastapi: The framework... uvicorn: ...a module to run our application' [Adeshina, p32].

- **requirements-api.txt = -r requirements.txt + fastapi + uvicorn**

A mesma regra do requirements-coleta.txt: o app publicado não importa o FastAPI.

E o uvicorn se declara mesmo vindo com o Streamlit: a API o usa diretamente.

- **Não copie as versões do livro (fastapi==0.78.0 [p194])**

Elas trazem Starlette 0.19 e Pydantic 1; o Streamlit 1.61 exige Starlette ≥ 0.46.

No mesmo ambiente, a API de 2022 quebraria o painel de 2026 (medido).

- **Nem 'fastapi[standard]'**

Traz o comando 'fastapi dev' e uma dúzia de pacotes que a API não usa.

► O que se importa, se declara — no arquivo do ambiente que importa. Pela terceira vez.

A menor API

passo01_ola.py

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
# o decorador diz QUAL pedido a função atende: GET em "/"
```

```
@app.get("/")
```

```
def ola():
```

```
    return {"mensagem": "Olá! Esta é a API do Painel ODS Brasil."}
```

```
# o dicionário devolvido VIRA JSON
```

Ler o comando, peça por peça

`uvicorn passo01_ola:app --reload → http://127.0.0.1:8000`

- **passo01_ola:app** — 'no módulo `passo01_ola`, a variável `app`'
'file:instance' [Adeshina, p33]. No projeto: `uvicorn api.main:app` (o pacote `api`, o arquivo `main`).
- **--reload** — reinicia o servidor a cada `.py` salvo
Só no desenvolvimento. E só funciona num terminal de verdade (VS Code, PowerShell).
- **Sem --port nem --host: porta 8000, endereço 127.0.0.1**
Abra `/` (o JSON), `/docs` (a documentação interativa [p49]) e `/redoc` [p51].

► **Ao vivo: subam o passo 1 e abram o `/docs`. Ninguém escreveu aquela página.**



python main.py — e nada aconteceu

Tropeço real da v9: o primeiro arquivo da API, rodado como tudo desde a Aula 1.

- **> python api/main.py → ~2 s, código 0, nenhuma mensagem — e nenhum servidor**

O arquivo só DEFINE a aplicação (a variável app). Quem a SERVE é o uvicorn.

- **O mesmo vale para o app: ele não roda com python app.py**

Roda com streamlit run. A API, com uvicorn.

- **E de dentro de api/: ModuleNotFoundError: No module named 'src'**

A API importa src/; o ponto de partida é a pasta do projeto — como no python -m da Aula 10.

▶ Definir não é servir. O servidor é um programa à parte: streamlit run, ou uvicorn.

Os quatro sintomas de ambiente

Subcompetência 3.3 na prática: quando a API não sobe, é quase sempre um destes.

- **python arquivo.py termina calado**

O Python não é servidor → `uvicorn arquivo:app --reload`.

- **'Could not import module "main"' / 'No module named src'**

Rodou da pasta errada → `cd` para a pasta do arquivo; no projeto, a pasta do projeto.

- **[Errno 10048] error while attempting to bind on address... :8000**

Já há um servidor na porta — às vezes um ÓRFÃO de uma janela fechada 'no X' (visto aqui).

`Ctrl+C` no terminal antigo; ou `netstat -ano | findstr :8000` e `taskkill /PID <n> /F`.

- **/docs em branco**

O navegador está sem internet (próximo slide). A API está bem.

► Pare o servidor com `Ctrl+C`, sempre. É o `finally` da API.

Dois avisos de sala

Os dois parecem defeito da API. Nenhum é.

- **O /docs precisa de internet — no NAVEGADOR**

A página carrega o Swagger de cdn.jsdelivr.net. Sem acesso externo: em branco (medido).

A API continua respondendo: teste pela URL da rota, ou pelo passo 4.

- **No PowerShell 5.1, curl NÃO é o curl**

É apelido do Invoke-WebRequest, e mostra 'olá' como 'olÃ¡'. Use curl.exe, o navegador ou o /docs.

- **E o curl `http://0.0.0.0:8080/` do livro [p33]**

No Windows, 0.0.0.0 não é endereço de destino (WinError 10049). Use 127.0.0.1 ou localhost.

► **Antes de 'consertar' a API, confira quem está lendo a resposta.**



3 · Rotas que respondem

Sem tipo × com tipo

passo02_tipos.py

```
@app.get("/sem_tipo/ranking/{ano}")
def ranking_sem_tipo(ano):
    return [d for d in DESMATAMENTO if d["ano"] == ano]
```

```
@app.get("/ranking/{ano}")
def ranking(ano: int = Path(ge=1988)):
    return [d for d in DESMATAMENTO if d["ano"] == ano]
```

```
# prevejam: /sem_tipo/ranking/2024 e /ranking/2024
```

O tipo é a guarda

Tudo o que chega numa URL é TEXTO. '2024', no endereço, é a palavra — não o número.

- **Sem tipo: /.../ranking/2024 → 200 e lista VAZIA (ano chegou como str)**
E abc, 1950 — tudo 200 vazio. Nunca dá erro; nunca dá dado. Tropeço real da v9.
- **Com tipo: 2024 → 9 UFs · abc e 1950 → 422, com o motivo**
'Input should be a valid integer' · 'greater than or equal to 1988'.
- **uf: UfAmazonia (um Literal com as 9 UFs)**
ZZ e pa → 422 com a lista das nove — e, no /docs, um MENU com as nove.

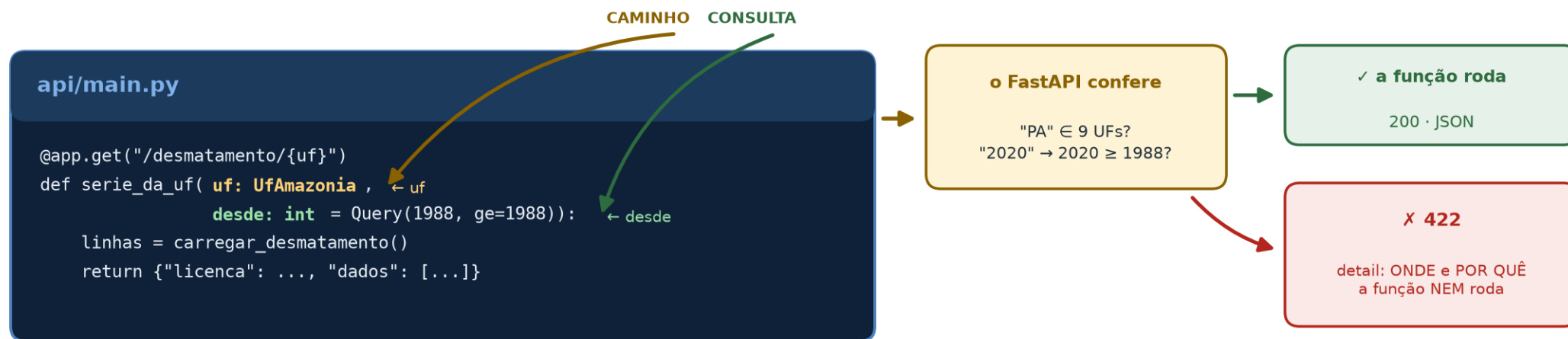
▶ **Sem tipo, a API ENGANA (200 vazio). Com tipo, ela QUEBRA (422, com o motivo). Prefira o que quebra.**

Anatomia de uma rota

Entre { }: do CAMINHO. O resto: da CONSULTA. Os tipos são conferidos antes de a função rodar.

Anatomia de uma rota: do endereço à função — e de volta, em JSON

GET http://127.0.0.1:8000/desmatamento/PA?desde=2020



Entre { }: do CAMINHO. O resto: da CONSULTA. O tipo converte o texto da URL — e recusa, antes da função, o que não converte.

Caminho × consulta

Passo 3, parte A: /desmatamento → 36 linhas · ?desde=2024 → 18 · ?desde=2024&limite=3 → 3.

- **O QUE você quer vai no CAMINHO**

A UF da série, o ano do ranking: /desmatamento/PA · /desmatamento/ranking/2024.

- **O COMO você quer vai na CONSULTA**

Desde quando, até quando, quantas linhas: ?desde=2020&ate=2024&limite=5.

- **'A query parameter is an optional parameter' [Adeshina, p47] — só com valor padrão**

def f(secao: str) sem ?secao= → 422 'Field required' (verificado). Opcional: secao: str | None = None.

► **Identifica o recurso? Caminho. Recorta a resposta? Consulta.**



A rota que engolia a outra

Tropeço real da v9: /desmatamento/{uf} declarada ANTES de /desmatamento/total.

- **GET /desmatamento/total → 422 'Input should be AC, AM, ...' ('input': 'total')**

O FastAPI testa as rotas na ORDEM em que foram declaradas; {uf} casa com qualquer palavra.

A rota do total nunca era alcançada.

- **Quem avisou foi o TIPO**

Com uf sem tipo, 'total' viraria uma UF inexistente: 200 e lista vazia.

- **O livro não trata da ordem**

É o tipo de coisa que só aparece rodando — passo 3, parte B.

► **Caminho FIXO antes de caminho com PARÂMETRO.**

Livro × hoje — o FastAPI também mudou

O livro é de 2022. Cinco divergências, ensinadas e não escondidas.

- **async def em tudo, sem explicação [p32]** · só o Unix: `source venv/bin/activate` [p26]
No curso: `def` (medido) e o Windows desde a Aula 5 (`.venv\Scripts\Activate.ps1`).
- **`curl http://0.0.0.0:8080/` depois de subir na porta 8000 [p32–33]**
A errata da editora (Packt) manda usar a 8000; e no Windows 0.0.0.0 nem é destino.
- **Versões fixadas (`fastapi==0.78.0` [p194]) · `query 'optional'` [p47] · `'le = less than'` [p47]**
Quebram o Streamlit · só com padrão · é 'menor OU IGUAL'. E a ordem das rotas: nem citada.

► Quarta vez no semestre (@st.cache, pages/, Selenium, e isto): o livro não está errado — está anterior.

A17 · o código (sobe sem erro)

api/main.py · a primeira versão de um colega

```
app = FastAPI()

@app.get("/desmatamento/{uf}")
def serie(uf):
    return [l for l in carregar_desmatamento() if l["uf"] == uf]

@app.get("/desmatamento/total")
def total():
    return total_por_ano(carregar_desmatamento())

@app.get("/desmatamento/ranking/{ano}")
def ranking(ano):
    return ranking_do_ano(carregar_desmatamento(), ano)
```

O /docs da v9

Captura real: sete rotas em três grupos — gerado do código, com as docstrings como descrição.

API do Painel ODS Brasil v9 — Aula 11 OAS 3.1

/openapi.json

Os dados que o **Painel ODS Brasil** coleta, para outros programas: o desmatamento anual por UF da Amazônia Legal (INPE/PRODES, **CC BY-SA 4.0**) e as notícias de meio ambiente da Agência Brasil (**CC BY 3.0 BR**). Toda resposta traz a fonte e a licença — quem reutilizar deve citá-las (e, no caso do INPE, compartilhar igual).

sobre

GET / `Apresentacao`

GET /fontes Fontes

desmatamento

GET /desmatamento Desmatamento

GET /desmatamento/total Desmatamento Total

GET /desmatamento/ranking/{ano} Ranking

GET /desmatamento/{uf} Serie Da Uf

noticias

GET /noticias Noticias

Schemas

HTTPValidationError > Expand all `object`

O outro lado: o código da Aula 2, apontado para nós

passo04_cliente.py · com a API no ar

```
import requests

r = requests.get("http://127.0.0.1:8000/desmatamento/PA",
                 params={"desde": 2020}, timeout=10)
r.raise_for_status()
for linha in r.json()["dados"]:
    print(linha["ano"], linha["area_km2"])

# 2020 4899.0 · 2021 5238.0 · ... · 2025 2064.0      (~25 ms)
# um 422 visto de fora: detail = ['query', 'desde'] + o motivo
```

A licença viaja com o dado

Toda rota de dados da v9 devolve um envelope — não uma lista solta.

- `{"fonte": ..., "licenca": "... (CC BY-SA 4.0)", "coletado_em": ..., "linhas": 3, "dados": [...]}`

Quem recebe só os números não tem como saber que deve citar — e compartilhar igual.

- **As mesmas funções de src/, os mesmos arquivos**

O que a API serve é, por construção, o que o painel mostra.

- **E 'não encontrado'? ranking/2030 → 200 com 'linhas': 0**

O livro faz igual no cap. 2 [p45] — e admite no cap. 3: 'a 200... instead of a 404' [p66]. Etapa 7.

- ▶ **A licença é parte da resposta. E 200 ainda não quer dizer 'achei' — isso se corrige na Etapa 7.**

Ao vivo: a v9 em dois terminais

terminal

terminal 1 – o painel (pessoas)

streamlit run app.py # :8501

terminal 2 – a API (programas), da pasta do projeto

uvicorn api.main:app --reload # :8000/docs

git add .

git commit -m "v9: API do projeto com FastAPI (7 rotas GET)"

git push # a pasta api/ sobe; o Cloud segue rodando só o app

O TP3 depois de hoje

O item 4 está pela metade — e a metade que falta tem data.

-  **Itens 1–3 — documentação, páginas, coleta (Aulas 9 e 10)**
Revisão v9 do Charter: um público novo — PROGRAMAS.
-  **Item 4, consulta — ambiente + rotas GET, documentadas no /docs**
Sete rotas na v9; o TP3 pede ao menos duas.
-  **Item 4, envio — POST, com validação**
Aula 12 — e a documentação de cada rota melhora lá.
-  **Item 5 — preparar para LLMs**
A Aula 12 começa a receber TEXTO pela API.

► [ASK] No SEU projeto: que pergunta merece um endereço?

Anti-padrões de uma API

Quase todos respondem 200.

- **Rota sem tipo · caminho com parâmetro antes do caminho fixo**
200 e lista vazia — o consumidor conclui que não há dados.
- **async def com trabalho que bloqueia · copiar as versões do livro**
Pedidos em fila (8,0 s × 2,0 s); Starlette e Pydantic antigos quebrando o Streamlit.
- **Resposta sem licença · a API lendo os dados do seu próprio jeito**
CC BY-SA ignorado; painel e API mostrando números diferentes.
- **fastapi no requirements.txt de um app que não o importa**
Peso morto no build do Community Cloud.

Princípios que levamos

Quatro frases que sobrevivem à próxima versão do FastAPI.

- **O painel é para pessoas; a API, para programas**

Oferecer o degrau 1 é melhor do que obrigar alguém a raspar.

- **O tipo é a guarda**

Tudo o que chega numa URL é texto.

- **O QUE no caminho; o COMO na consulta**

E caminho fixo antes de caminho com parâmetro.

- **A licença viaja com o dado**

Quem distribui herda o dever de citar.



O caso, em 3 lições (v9)

O que a v9 ensinou, na ordem em que doeu.

- **O painel é para pessoas; a API, para programas**

Para um programa, o painel é uma casca: 'You need to enable JavaScript'.

- **O tipo é a guarda**

ranking/2024 sem tipo: 200 e vazio. Com tipo: 9 UFs — e 422 para o resto.

- **A ordem e o endereço são decisões**

/desmatamento/total, engolida por {uf}, até mudar de lugar.

► **Aula 12: as equipes de campo querem MANDAR relatos. Uma API que recebe precisa decidir o que aceita.**